UrbanPulse on Azure

Week 1 Summary: what was built, why, and what it means - explained in plain language

Author	Saurabh Nikam
Period	Week 1: 26 to 31 October 2026 (Days 1 to 7 of a 28-day cloud window)
Project code	github.com/SaurabhNikam095/urbanpulse-azure-lakehouse (release v0.1.0)
Status	Foundations complete. Nothing is deployed to the cloud yet; the cloud build starts in Week 2.

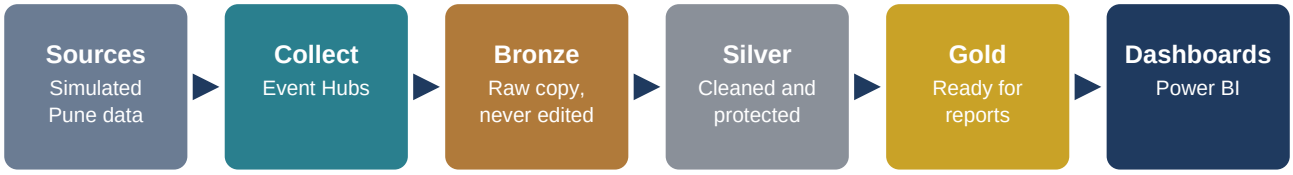
The project in plain words

Imagine a ride-hailing company like Ola or Uber. Every second, thousands of small pieces of information are created: a rider asks for a trip, a driver accepts, a phone sends its location, a payment goes through. On their own these are just noise. **UrbanPulse is the system that collects all of this information, cleans it, protects people's privacy, and turns it into reports** that managers can trust - for example, 'Where is demand higher than the number of drivers right now?' or 'How much money did we make yesterday?'

Think of it as a **water supply plant**. Water (data) arrives from many rivers, some of it muddy. The plant filters it in stages, tests it, and delivers clean water to homes. If the plant breaks, nobody notices until the taps run dry - so the plant must be reliable, monitored and safe. This project is about building that plant properly.

Because a real company's data is private, the project **generates realistic, made-up rides for the city of Pune**. No real person's information is used anywhere.

How the data flows



Data is stored in three stages, called **Bronze, Silver and Gold**. Bronze is the raw delivery, kept exactly as it arrived so mistakes can always be replayed. Silver is the cleaned version (duplicates removed, private details protected). Gold is the polished version shaped for reports and dashboards.

Week 1 at a glance

7	129	4	Green	v0.1.0
days of work	automatic tests passing	written design decisions	checks on GitHub	first tagged release

Why this project exists

It is built to look and behave like a real company's data platform, so every part answers a question a business would actually ask:

Who needs it	What they want to know	What UrbanPulse gives them
Operations team	Is demand higher than the number of drivers? Should prices rise?	A live picture of demand and supply per area of the city
Finance team	How much did we earn? How many payments failed or were cancelled?	Daily revenue and cancellation reports
Risk and fraud team	Are there suspicious trips or payments?	Flags for unusual activity
Executives	How is the business doing this week? Are drivers staying?	A weekly summary dashboard
Data science team	Clean, consistent data to train models	Ready-to-use tables and a simple price-surge model

The cloud platform used is **Microsoft Azure**. The work is planned inside a **28-day window** of cloud access, so the plan is deliberately organised: do all the safe preparation on a laptop first (cheap and risk-free), then use the cloud in focused bursts and switch it off after each session.

Phase	Focus	Status
Week 1 (Days 1-7)	Foundations: tools, data generators, data rules, cleaning logic, privacy, cloud blueprints	**Done**
Week 2 (Days 8-14)	First real cloud build; send live data into the cloud; store the raw copy; capture database changes	Next
Week 3 (Days 15-21)	Large-scale cleaning, business-ready tables, scheduled jobs, reprocessing history	Planned
Week 4 (Days 22-28)	Quality checks, monitoring, performance tests, failure drills, dashboards, switching everything off, final documents	Planned

Day by day: what was done in Week 1

Day 1	Setting up a safe, professional workshop
What we did	Set a **spending alert** on the cloud account before creating anything. Created the project's home on **GitHub** (a public website for storing code and its history), organised the folders, and installed the tools. Added automatic checks that run before every save, including one that **blocks passwords from being stored by accident**.
Why it matters	Cloud services bill by the hour, so cost control comes first. A tidy, automatically checked project is trusted more than a messy one.
Proof	Budget alert screenshot; project history visible on GitHub.

Day 2	Choosing the cloud tools and creating the first data generator
What we did	Read Microsoft's documentation to choose the right level of the messaging service (**Event Hubs Standard**, 2 units) and the region (**Central India, Pune**). Wrote the choice and the reasons down in a short design note (ADR-001). Built the **trip simulator**: it creates rides from request to completion, with about 12% cancelled.
Why it matters	The design note records *why* a choice was made and what was rejected, which is what real engineering teams do. The simulator gives safe, repeatable test data.
Proof	ADR-001 on GitHub; 16 automatic tests.

Day 3	Making the data messy on purpose
What we did	Added GPS location signals from 500 drivers every 5 seconds and payments linked to completed trips (with **fake** phone numbers). Then added deliberate **mistakes**: about 2% of records duplicated, 3% arriving 1 to 10 minutes late, 1% broken (for example a missing location or a negative fare). A **log records exactly what was damaged**.
Why it matters	Real data is never perfect. A system that is only tested on clean data fails in real life. The log lets us prove later that every problem was caught.
Proof	46 automatic tests passing.

Day 4	Agreeing on the shape of the data, with automatic checking
What we did	Wrote **data contracts**: precise descriptions of what a trip, a GPS signal and a payment must look like (trips have a version 1 and a version 2 that adds a surge-price field). Built a script that checks data against them, and set up **GitHub Actions**: a robot that runs all checks and tests every time new work is uploaded.
Why it matters	When many systems share data, an agreed format prevents silent breakage, like a standard form. The robot catches mistakes within minutes, before anyone depends on them.
Proof	Green checkmark on GitHub for every upload.

Day 5	Cleaning the data
What we did	Built the first stages of the pipeline. **Bronze** keeps a raw copy with its origin details. **Silver** removes duplicates and handles late arrivals using a **5-minute time limit** (called a watermark). Records later than that are **kept and marked, not thrown away**. This policy is written up in ADR-002.
Why it matters	Duplicates would count the same payment twice and inflate revenue. Throwing away late records would lose real money. Both must be handled deliberately.
Proof	Automatic tests for each rule; ADR-002 on GitHub.

Day 6	Protecting privacy and remembering history
What we did	Phone numbers and emails are replaced with **irreversible scrambled codes** (using a secret ingredient called a salt), so analysts can still tell that two records belong to the same person but can never read the real details. Also built **SCD Type 2**: when a driver or rider changes (for example a new phone), the old record is closed and a new one opened, so history is kept. It copes with duplicate, unchanged and out-of-order updates.
Why it matters	Privacy laws and common sense require that sensitive details are protected. Keeping history means we can answer 'what was true last month?'
Proof	129 automatic tests passing; sample change file used in tests.

Day 7	Planning the cloud and closing the week
What we did	Chose **Azure Databricks** as the engine for large-scale processing (ADR-003) and **Azure Data Factory** for scheduling jobs (ADR-004). Wrote **Terraform** 'blueprints' for the first cloud pieces: a resource group, a data storage lake, a secrets vault and the messaging service. They were checked but **not built**. Reviewed the week, and published release **v0.1.0**.
Why it matters	Describing infrastructure as reviewable files means the whole environment can be created, and destroyed, in minutes, with no manual clicking and no forgotten leftovers.
Proof	ADR-003 and ADR-004; Terraform checks passing; v0.1.0 tag; CI screenshot.

Key decisions and the reasons

Decision	Choice	Plain-language reason
Messaging service	Event Hubs Standard, 2 units	The cheapest level that supports the needed features; enough for the planned load
Region	Central India (Pune)	Closest to the user, and has several data-centre zones
Database-change capture	Merge changes into one stream	The chosen tier allows only 10 channels; this keeps usage at 8
Late data	Keep and mark, never drop	Nothing is silently lost; late records can still be analysed
Processing engine	Azure Databricks	Chosen in ADR-003 for large-scale processing
Scheduling	Azure Data Factory	Chosen in ADR-004 to run jobs on a schedule
Privacy	Hash phone and email	Real contact details never reach reports or dashboards

Every decision above is written in a short **ADR (architecture decision record)** stored with the project, so anyone can see what was chosen, when, and why.

Problems met and how they were solved

What went wrong	What it taught
A placeholder email was used for the project history, so work would not appear under the right profile	Always check a value before trusting it; fixed and re-checked
Commands were run from the wrong folder	Check where you are before you run anything
The automatic checks stopped a save to fix formatting	That was the safety net doing its job; the fix was to save again
A sample data file had been forgotten, so the online checks failed even though everything worked on the laptop	'Works on my machine' is not enough; the online robot found it, and adding the file fixed it

Plain-English glossary

Term	Meaning
Cloud / Azure	Renting computers and services over the internet instead of owning them; Azure is Microsoft's
Data pipeline	A chain of steps that moves data from where it is created to where it is used, cleaning it on the way
Streaming	Handling data continuously as it arrives, not once a day
Event	One small record of something that happened, such as 'trip completed'
Simulator	A program that creates realistic made-up data for testing
Duplicate	The same record received twice, for example after a sender retries
Late event	A record that arrives well after the moment it describes
Data contract	An agreement on exactly how a piece of data must be laid out
PII and hashing	Personally identifiable information (phone, email) and the one-way scrambling that protects it
SCD Type 2	A way of keeping the full history of changes to a person's or object's details
Automated test	A small program that checks another program still works
CI (continuous integration)	A robot that runs all checks every time new work is uploaded
Terraform	A tool that creates cloud resources from written instructions, repeatably
ADR	A short note recording a decision, the reasons, and the alternatives
Key Vault	A locked safe in the cloud for passwords and secrets

Skills practised in Week 1

- Cloud cost control (budgets and alerts) and planning around a fixed time window
- Version control with Git and GitHub: small, well-described commits and releases
- Python programming with automated testing (129 tests) and code-quality tools
- Data design: schemas and data contracts, handling duplicates and late data
- Privacy and security: secret scanning, hashing of personal data, no secrets in code
- Infrastructure as code with Terraform, and recording decisions in ADRs

What comes next

Week 2 begins with the first real cloud build. The prepared blueprints will be reviewed with their cost estimate, applied, and checked in the Azure portal. Then the simulators will send events live into the cloud, and everything will be switched off again at the end of each session to protect the budget.

Honest status: at the end of Week 1, no cloud resources have been created, and every number in this document comes from tests and checks that can be re-run from the project on GitHub. Performance and cost figures will be added only after they are measured in the cloud.